

Q1.

a) O compilador vai gerar um erro de compilação e o código simplesmente não vai rodar. Isso acontece porque uma classe abstrata serve como um molde para as outras classes. Sendo assim, um método abstrato é apenas uma assinatura, ele não possui nenhuma implementação ou corpo na classe pai. A responsabilidade de implementar a lógica real é das classes filhas, que fazem isso obrigatoriamente através do @Override.

b) Faz sentido porque ao definir o modificador como protected, o Java restringe o acesso, permitindo o uso apenas dentro das próprias subclasses e por outras classes que pertençam ao mesmo pacote. Isso protege o uso, garantindo que ele não fique exposto para o resto do sistema, mas que fique disponível para quem precisa dele para funcionar.

c) super(...) serve para chamar diretamente o construtor da classe pai. Sem ele, o processo de instanciação do objeto estaria incompleto, pois a classe filha depende dos atributos e da inicialização que a classe pai faz. O super garante que a base do objeto seja criada antes que a subclasse adicione seus atributos próprios.

---

Q2.

a) O atributo está marcado como final para impedir que ele seja alterado após a sua inicialização. Isso faz com que o valor atribuído a ele no momento da criação do objeto seja único e imutável, funcionando como uma constante.

b) A data e o horário são definidos com base na hora local do sistema onde o código está rodando. Esse valor exato é capturado e setado de forma automática e dinâmica no momento em que o objeto for instanciado.

c) Caso mudasse para public, o encapsulamento seria quebrado. Tudo no sistema passaria a ter acesso direto a esse atributo. Isso significa que qualquer outra classe ou parte do código poderia alterar o valor da variável a qualquer momento, gerando falta de segurança e inconsistência nos dados.

---

Q3.

a) Isso acontece durante o tempo de execução, através do polimorfismo. Se você instanciar um Carro que é do tipo Veiculo, o Java resolve qual método chamar dinamicamente. Seria meio que fazer a referência genérica "veiculo." se transformar na execução do comportamento do "carro.", chamando o método sobrescrito.

b) Essa mecânica traz grandes vantagens para o projeto, ela reduz a quantidade de linhas de código , facilita a manutenção do sistema e melhora a legibilidade.

c) Não, não precisaria de nenhuma alteração na assinatura do método. Como a classe Caminhonete seria uma subclasse de Veiculo, ela seria considerada do tipo Veiculo. assim, o método existente conseguiria receber e processar o objeto.

---

Q4.

a) Existem diferenças na estrutura e na semântica. Uma interface suporta herança múltipla, enquanto uma classe abstrata permite apenas a herança única. Na semântica, a interface funciona como um "funciona como / é capaz de", enquanto a classe abstrata define o que o objeto realmente "é um".

b) Porque, desse jeito, a interface Repositorio passa a funcionar como um contrato de capacidade, especificando quais operações de banco de dados devem existir. Ao adicionar o uso de Generics com o <T>, ela se transforma em um "contrato template" reaproveitável, que pode ser utilizado de diferentes formas para qualquer entidade do sistema, sem precisar criar uma interface nova para cada classe.

c) Essa sintaxe aplica uma restrição estrita no uso do Genérico. Isso impede que o tipo <T> seja qualquer classe aleatória do sistema, forçando o Java a aceitar apenas a classe Entidade ou qualquer subclasse que herde diretamente dela.

---

Q5.

a) Para aplicar o conceito de Injeção de Dependências. Ao passar a interface no construtor em vez de instanciar uma classe concreta lá dentro, nós desacoplamos o código. Isso permite reutilização e flexibilidade, pois a classe passa a aceitar qualquer repositório que siga aquela interface,.

b) As classes VeiculoServico e VeiculoRepositorioEmMemoria não mudariam. O trabalho seria apenas criar a nova classe concreta chamada VeiculoRepositorioInArquivo e, depois disso, alterar apenas a classe Main para injetar essa nova implementação de arquivo na hora de dar o new VeiculoServico(...).

c) Ela afirma categoricamente que as regras de negócio de alto nível (como o VeiculoServico) nunca devem depender de mecanismos de baixo nível e detalhes de armazenamento (como o repositório em banco, memória). Ambos os lados devem depender de abstrações,

tornando o sistema modular e livre.